Computer Programming 1

Types – part 1

(14-09-2026)

Outline

- Variables and Constants
- Fundamental data types
  - Boolean
  - Integer
  - Real
  - Enum
  - Char

Outline

- Variables and Constants
- Fundamental data types
  - Boolean
  - Integer
  - Real
  - Enum
  - Char

Variables and constants

- To store values in a memory, entities called variables or constants are used
- Variables allow the modification of their values
- The corresponding memory area is identified by a name, which identifies the memory address

A variable is characterized by:

- Name (identifier)
- Type
- Memory location (I-value or address)
- Value (r-value)

Definition and declaration of variables (1)

- Definition
  - Syntax: type identifier;
  - E.g., int x;
  - If a variable is defined outside of any function, (main() included), it is a global variable and it is automatically initialized to 0
- Definition with initialization
  - Syntax : type identifier=exp;
  - E.g., int x=3;
  - E.g., int x=3*2;
  - E.g., int y=3*x; //x has been previously defined
  - E.g., ~~int y=3*b;~~ //b has not been previously defined
- Declaration
  - Syntax : extern type identifier;
  - E.g., extern int x;

Variable output

The following instruction is used to print to output the value of a variable x :

```
cout << x;
```

Definition and declaration of variables (2)

- Definition: when the compiler encounters a definition of a variable, it prepares the allocation of a memory area capable of containing a variable of chosen type
- Declaration: specifies only the type of the variable and therefore assumes that the variable is defined in another part of the program

E.g.,

int x; → definition
int x=2; → definition with initialization
extern int x; → declaration

Example of variable definition

{../L3-4_EXAMPLE_TYPES/variables.cc}

Example with initialization

{../L3-4_EXAMPLE_TYPES/variables2.cc}

Definition and declaration of variables (2) - examples

```
using namespace std;
#include <iostream>

int main ()
{
  int n=10;                 // initialization
  float pi = 3.1415;        // initialization
  float twopi = pi*2;       // initialization
  char c = 'C';             // initialization

  cout << "n = " << n << endl;
  cout << "pi = " << pi << endl;
  cout << "twopi = " << twopi << endl;
  cout << "c = " << c << endl;
  return 0;
}
```

```
$ g++ variables.cc
$ ./a.out
n = 10
pi = 3.1415
twopi = 6.283
c = C
```

{../L3-4_EXAMPLE_TYPES/variables2.cc}

Definition and declaration of variables (2) - examples

```
using namespace std;
#include <iostream>

int main ()
{
  int n;
  float pi;
  float twopi = pi*2;   // initialization
  char c;

  cout << "n = " << n << endl;
  cout << "pi = " << pi << endl;
  cout << "twopi = " << twopi << endl;
  cout << "c = " << c << endl;
  return 0;
}
```

Always initialize the defined variable

```
$ g++ variables.cc
$ ./a.out
n = 32767
pi = 0
twopi = 0
c = 3
```

{../L3-4_EXAMPLE_TYPES/variables.cc}

Declaration of constants

Syntax:

const type identifier = exp;

- exp must be an expression whose value must be computed during the program compilation phase
- Sometimes (depending on the compiler) it is possible to initialize the constants to non-constant values, but the result cannot be determinate a-priori → avoid this!

E.g.,

```
const int kilo = 1024;
kilo = 1023;  //error, the value cannot be changed
const double pi = 3.14159;
const int oneThousand = kilo - 24;
```

Example of constant use

{../L3-4_EXAMPLE_TYPES/costants.cc}

Outline

- Variables and Constants
- Fundamental data types
  - Boolean
  - Integer
  - Real
  - Enum
  - Char

Type

- Types are sets of objects of the same type
- Use the same space in memory and the same encoding
- Are subject to the same operations, with the same meaning
- Advantages of using types
  - Semantic correctness
  - Efficiency in memory usage, due to the knowledge of the required memory at compile time

Fundamental and derived types in C++

Two types:

- Fundamental types
  - They are used to represent simple information

    E.g., integer numbers (int), characters (char)
- Derived types (not today)
  - They allow constructing complex data structures
  - They are based on fundamental data
  - They use specific constructors (array, pointers, etc.)

Fundamental types

- **Integer:** short, int, long, long long
- **Boolean:** bool
- **Enumeration:** enum
- **Character:** char
- **Real:** float, double, long double

Note

- Integer, Boolean, Enumeration and Character have a finite/closed domain (they are called discrete)

Integer types (signed)

- Signed integer types consist of integers ranging between two extreme values, depending on the implementation
  - Typically, they are encoded in 2-complement with N bit (N=16,32,…)
  - They ranges from $[-2^{N-1}, 2^{N-1}-1]$
- Four types of increasing size exist
  - short [int], int, long [int], long long [int]
- Their size depends on the implementation (PC, OS, etc.)
  - sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)
  - Typically
    - short is 16 bits [-32768; 32767] (at least 16bits by the standard)
    - int is 32 bits [-2147483648; 2147483647] (at least 16bits by the standard)
    - long is 64 bits [-9223372036854775808; 9223372036854775807] (at least 32bits by the standard)
    - long long is 64 bits [-9223372036854775808; 9223372036854775807] (at least 64bits by the standard)
- The value of an entire expression is in the range $[-2^{N-1}, 2^{N-1}-1]$ since values are cycling, e.g., after 2147483647 we have -2147483648

sizeof(t) returns the dimension of the type in bytes

for short: 32767+1= -32768

Example of short, int, long, long long:

{ ../L3-4_EXAMPLE_TYPES/shortlong.cc}

Integer types (unsigned)

- Unsigned types represent non-negative integers of various size
  - Encoding unsigned integers with N bits (N=16,32,…), range [0, 2^N-1]
  - Typically, not used

```
unsigned int x= 1232;
unsigned short x= 567;
unsigned long x= 878678687;
```

- The value of an entire expression is in the range [0, 2^N-1]
  - if the arithmetic value is outside this range, the returned value is *X module* 2^N

Example of unsigned:

{ ../L3-4_EXAMPLE_TYPES/unsigned.cc}

Arithmetic operators for integers

Binary operator	Meaning
+	Addition
-	Subtraction
*	Multiplier
/	Integer division
%	Remainder for the integer division
Unary Operator	**Meaning**
-	Sign inversion

The division is integer division
E.g., 5/2 = 2 not 2.5

The reminder is, e.g.,
- 5%2 = 1
- 4%2 = 0

Examples on arithmetic operators on integers:
{ ../L3-4_EXAMPLE_TYPES/operators_arithmetic.cc}

Bit-to-bit operators (unsigned integer)

Operator	Example	Meaning
>>	x>>n	Right shift of n positions
<<	x<<n	Left shift of n positions
&	x&y	bit-to-bit AND between x and y
|	x|y	bit-to-bit OR between x and y
^	x^y	bit-to-bit XOR between x and y
~	~x	NOT, bit-to-bit complement. Note: it returns a signed integer (in the 2-complement representation), even if the input is unsigned

Note: integers are converted to binary before the operator's application

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:     0000000000001100    (12)
y:     0000000000001010    (10)
```

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:      0000000000001100    (12)
y:      0000000000001010    (10)
x|y:    0000000000001110    (14)
```

	OR	AND	XOR
00	0	0	0
01	1	0	1
10	1	0	1
11	1	1	0

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:      0000000000001100    (12)
y:      0000000000001010    (10)
x|y:    0000000000001110    (14)
x&y:    0000000000001000    ( 8)
```

	OR	AND	XOR
00	0	0	0
01	1	0	1
10	1	0	1
11	1	1	0

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:      0000000000001100    (12)
y:      0000000000001010    (10)
x|y:    0000000000001110    (14)
x&y:    0000000000001000    ( 8)
x^y:    0000000000000110    ( 6)
```

	OR	AND	XOR
00	0	0	0
01	1	0	1
10	1	0	1
11	1	1	0

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:     0000000000001100    (12)
y:     0000000000001010    (10)
x|y:   0000000000001110    (14)
x&y:   0000000000001000    ( 8)
x^y:   0000000000000110    ( 6)
~x:    1111111111110011    (65523 or-13 )
```

Note: 65523 in 2-complement is -13

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:     0000000000001100    (12)
y:     0000000000001010    (10)
x|y:   0000000000001110    (14)
x&y:   0000000000001000    ( 8)
x^y:   0000000000000110    ( 6)
~x:    1111111111110011    (65523 or-13 )
x>>2:  0000000000000011    ( 3)
```

Note: 65523 in 2-complement is -13

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Example: bit-to-bit operators

- Given x and y represented with 16 bits

```
x:     0000000000001100    (12)
y:     0000000000001010    (10)
x|y:   0000000000001110    (14)
x&y:   0000000000001000    ( 8)
x^y:   0000000000000110    ( 6)
~x:    1111111111110011    (65523 or-13 )
x>>2:  0000000000000011    ( 3)
x<<2:  0000000000110000    (48)
```

Note: 65523 in 2-complement is -13

Example of bit-to-bit operators on unsigned integer:

{ ../L3-4_EXAMPLE_TYPES/operators_bitwise.cc}

Assignments

- Syntax of the assignment is: exp1 = exp2
  - exp1 is an expression with an address (I-value)
  - exp1 and exp2 are of a compatible type
- The value of exp2 is evaluated and then assigned to exp1
- Example of assignment:

  { ../EXAMPLE_BASE/assignement_errors.cc}

- An assignment can occur inside another expression
  - The value of an assignment is the value of exp2
  - The assignment operator, '=', is associated to the right
  - E.g.,

    a = b = c = d = 5;

    It is equal to: (a=(b=(c=(d=5))));
  - Example of multiple assignment:

    { ../L3-4_EXAMPLE_TYPES/assignements.cc}

Relevance of the first assignment (initialization)

```
int a=12;
int b;        //Attention:not controlled initialization!
if(a==b){
        cout << "a==b" << endl;
else
        cout << "a!=b" << endl;  // a=12 b=0
}
```

Mixed operators with assignments and arithmetics

Extended	Short
x = x + y	x += y
x = x - y	x -= y
x = x * y	x *= y
x = x / y	x /= y
x = x % y	x %= y

The use of the short format is more efficient.

In some implementations it optimizes the use of the CPU

Examples on mixed assignment:

{ ../L3-4_EXAMPLE_TYPES/op_assignement.cc}

Increment and decrement operators

- x++
  - It increments x by one unit
  - It uses the value of x before the increment
- x--
  - It decrements x by one unit
  - It uses the value of x before the decrement
- ++x
  - It increments x by one unit
  - It uses the value of x after the increment
- --x
  - It decrements x by one unit
  - It uses the value of x after the decrement

Example of multiple assignment:

{ ../L3-4_EXAMPLE_TYPES/op_increment.cc}

E.g.,

```
x = 5;
x++;   //v is increased to 6
++x;   //v is increased to 6
```

…but…

Example 1

```
x =5;
cout << x++ <<endl; //5 is displayed, then x is increased to 6

x =5;
cout << ++x <<endl; //x is increased to 6 and then 6 is displayed
```

Example 2

```
x =5;
b = x++; //b is 5, then x is increased to 6

x =5
b = ++x; //x is increased to 6, then b assigned (b=6)
```

Increment and decrement operators - examples

```
using namespace std;
#include <iostream>

int main ()
{
 int x=0, y=2, z=3;

 cout << "(1) x = " << x << " y = " << y << " z = " << z << endl;
 x = z++;
 cout << "(2) x = " << x << " y = " << y << " z = " << z << endl;
 x = ++y;
 cout << "(3) x = " << x << " y = " << y << " z = " << z << endl;
 x = z--;
 cout << "(4) x = " << x << " y = " << y << " z = " << z << endl;
 x = --y;
 cout << "(5) x = " << x << " y = " << y << " z = " << z << endl;

 return 0;
}
```

```
$ g++ op_increment.cc
$ ./a.out
(1) x = 0 y = 2 z = 3
(2) x = 3 y = 2 z = 4
(3) x = 3 y = 3 z = 4
(4) x = 4 y = 3 z = 3
(5) x = 2 y = 2 z = 3
```

{../L3_01_EXAMPLE_BASE/op_increment.cc}

Evaluation order in an expression

- C++ does not specify the evaluation order in the expression
  - The order of subexpressions
  - The order of a function
- E.g., in the evaluation of the expression: exp1 <op> exp2, it is not specified if exp1 has to be evaluated before exp2 or vice-versa
  - (with some relevant exceptions, i.e., Booleans)

- It can represent a problem when the subexpression contain side-effect operators (e.g., increment operators)
  - E.g.,

```
j = ++x * x++;          //undefined behavior
v=a() + b() + c();      /* it is executed as: (a()+b())+c(), from left-to-right, since '+' requires left-to-right
                           associativity, but c() can be evaluated first, last or between a() and b()*/
```

  → Avoid using side-effect operators in subexpressions

For additional details:
http://en.cppreference.com/w/cpp/language/eval_order

Relationals operators

Operator	Meaning
==	Equal
!=	Different (not equal)
<=	Less than or equal to
>=	Greater than or equal to
<	Less than
>	Greater than

Boolean type

C++ provides the Boolean type: bool

- The value **false** is represented by a constant false (equivalent to 0)
- The value **true** is represented by a constant true (equivalent to any integer != 0)
- The type int can be used to the same objective
- Boolean operators are: ! (not), && (and), || (or)

x	y	!x	&&	||
false	false	true	false	false
false	true	true	false	true
true	false	false	false	true
true	true	false	true	true

!!x ⇔x,

!(x ||y)⇔ (!x && !y),

!(x && y)⇔ (!x || !y)

Lazy Evaluation



	OR	AND	XOR
00	0	0	0
01	1	0	1
10	1	0	1
11	1	1	0

- In C++ && and || are evaluated with the "lazy evaluation"
  - In (exp1 && exp2), if exp1 is evaluated false, then exp2 is not evaluated
  - Similarly, if exp1 is evaluated true in (exp1 || exp2)
  - The evaluation order between exp1 and exp2 is from left to right
- It is efficient
- Side effect if used with expression that can modify the value of the variables (e.g., "++")

  → Avoid using these constructors in Boolean expressions

Examples with the Boolean type

Example of booleans:

{ ../L3-4_EXAMPLE_TYPES/booblean_bol.cc}

Example of lazy evaluation with Boolean operators:

{ ../L3-4_EXAMPLE_TYPES/booblean_sideeffects.cc}

Real types

- Real types have a set of values composed of a subset of real numbers, that is, those representable within the computer in a predetermined format
- There are various types, in increasing order of precision
  - float (32bits, precision: 7 decimal digits)
  - double (64bits, precision: 15 decimal digits)
  - long double (96bits, precision: at least 15 decimal digits)
- Arithmetic operators: +, -, *, /
  - "/" between reals, e.g., $\frac{7.0}{2.0}$ =3.5
- Precision and memory allocation depend on the implementation

```
// creating float type variables
float num1 = 3.0f;
float num2 = 3.5f;
float num3 = 3E-5f; // 3x10^-5

// creating double type variables
double num4 = 3.0;
double num5 = 3.5;
double num6 = 3E-5; // 3x10^-5
```

```
// a double is assigned to a float variable
float a = 5.6;
```

Enum type

An enumerated type is a finite set of integer constants, defined by the programmer, each one identified by an identifier

- Syntax: enum typeID { id_or_int1, … , id_or_intn }
  - If not explicitly specified, values are equivalent to the integer 0, 1, 2, …
  - Only one value of the type enumerative can be assigned to an enum type variable
  - The values are printed as integers

Example of enum type:

{ ../L3-4_EXAMPLE_TYPES/enum.cc}

Enum- example (1/2)

```
using namespace std;
#include <iostream>

int main ()
{
 enum Day {MONDAY,TUESDAY,WEDNESDAY,THURSDAY,FRIDAY,SATURDAY,SUNDAY};
 Day today = MONDAY;
 //today = 3; //Error!
 cout << "today = " << today << endl;

  enum boolean { FALSE, TRUE};
  boolean X = FALSE;
  cout << "X = " << X << endl;

}
```

```
$ g++ enum.cc
$ ./a.out
today = 0
X = 0
```

{../L3_01_EXAMPLE_BASE/enum.cc}

Enum- example (2/2)

```
using namespace std;
#include <iostream>

int main ()
{
 enum color {RED=10, YELLOW=11, BLUE=12};
 color background= YELLOW, text= BLUE;
 cout << "background = " << background << endl;
 cout << "text = " << text << endl;

 enum color2 {ORANGE=13, GREEN, GRAY};
 color2 background2= ORANGE, text2= GREEN, text3= GRAY;
 cout << "background2 = " << background2 << endl;
 cout << "text2 = " << text2 << endl;
 cout << "text2 = " << text2 << endl;
}
```

```
$ g++ enum.cc
$ ./a.out
background = 11
text =12
background2 = 13
text2 = 14
text3 = 15
```

{../L3_01_EXAMPLE_BASE/enum.cc}

The character type

- The type char has the printable characters as value
  - E.g., ‘a’, ‘A’, ‘6’, ‘+’, ‘ ‘
  - Usually, a character requires 1 byte (8 bits)
- The char type is a subset of the type int
  - The numeric value associated with each character is named code and it depends on the used encoding
  - E.g., ASCII (the most used), EBCDIC, BCD, …

0	NUL	1	^A	2	^B	3	^C	4	^D	5	^E	6	^F	7	^G
8	^H	9	^I	10	^J	11	^K	12	^L	13	^M	14	^N	15	^O
16	^P	17	^Q	18	^R	19	^S	20	^T	21	^U	22	^V	23	^W
24	^X	25	^Y	26	^Z	27	^[	28	^\	29	^]	30	^^	31	^-
32	SP	33	!	34	"	35	#	36	$	37	%	38	&	39	’
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	|	125	}	126	~	127	DEL

ASCII table

- The type char, however, is not depending on the adopted encoding
  - The execution is not depending on the use encoding
  - Avoid referring to the ASCII value of a character
  - E.g.,
    - char c;
    - c=‘A’; //Yes
    - c=65; //Not use!

ASCII encoding

0	NUL	1	^A	2	^B	3	^C	4	^D	5	^E	6	^F	7	^G
8	^H	9	^I	10	^J	11	^K	12	^L	13	^M	14	^N	15	^O
16	^P	17	^Q	18	^R	19	^S	20	^T	21	^U	22	^V	23	^W
24	^X	25	^Y	26	^Z	27	^[	28	^\	29	^]	30	^^	31	^_
32	SP	33	!	34	"	35	#	36	$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	|	125	}	126	~	127	DEL

ASCII encoding

Alternative view

Dec	Hex	Alt	Esc	Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex	Character
0	0x00		Ctrl-@	NUL (Null)	32	0x20	[Space]	64	0x40	@	96	0x60	`
1	0x01	☺	Ctrl-A	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	☻	Ctrl-B	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	♥	Ctrl-C	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	♦	Ctrl-D	EOT	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	♣	Ctrl-E	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	♠	Ctrl-F	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	•	Ctrl-G	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	◘	Ctrl-H	BS Backspace	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	○	Ctrl-I	TAB \t	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	◙	Ctrl-J	LF Line Feed \n	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	♂	Ctrl-K	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	♀	Ctrl-L	FF Form Feed	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	♪	Ctrl-M	CR Carriage Return \r	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	♫	Ctrl-N	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	☼	Ctrl-O	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	►	Ctrl-P	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	◄	Ctrl-Q	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	↕	Ctrl-R	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	‼	Ctrl-S	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	¶	Ctrl-T	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	§	Ctrl-U	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	▬	Ctrl-V	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	↨	Ctrl-W	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	↑	Ctrl-X	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	↓	Ctrl-Y	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	→	Ctrl-Z	SUB (EOF)	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	←	Ctrl-[	ESC (Escape)	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	∟	Ctrl-/	FS	60	0x3C	<	92	0x5C	\	124	0x7C	|
29	0x1D	↔	Ctrl-]	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	▲	Ctrl-^	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	▼	Ctrl-_	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

https://ss64.com/ascii.html

Character encoding: general rules

All characters' encoding must follow the rules:

- Precedence
  - 'a' < 'b' < … < 'z'
  - 'A' < 'B' < … < 'Z'
  - '0' < '1' < … < '9'
- Consecutiveness between lowercase and uppercase letters and numbers
  - 'a' , 'b' , … , 'z'
  - 'A' , 'B' , … , 'Z'
  - '0' , '1' , … , '9'

Note

- The relation among lowercase and uppercase letters or between characters and numbers is not fixed

Character examples (1)

```
int main()
{
 char c1='A';
 cout << "c1="<< c1 << "\n";     //A
 //
 char c2=6;
 char c3='6';
 cout << "c2="<< c2 << " c3="<< c3 << "\n";   // c2=strange character     c3=6
 //
 int  in1=65;
 char c4 = in1;              // c4='A'
 int c5 = c4;                // c5=65
 if (in1 != c5)
   cout << "in1==c5 (in1=" << in1 << "  c5="<< c5 << " (c4="<< c4 << ")\n";
 else
  cout << "in1==c5  (c5="<< c5 << ")\n";            // c5=65
}
```

Character examples (2)

```
int main()
{
 char c1='A';
 cout << "c1="<< c1 << "\n";     //A
 //
 char c2=6;
 char c3='6';
 cout << "c2="<< c2 << " c3="<< c3 << "\n";   // c2=strange character     c3=6
 //
 int  in1=20001;
 char c4 = in1;                 // c4=strange character
 int c5 = c4;                   // c5=int derived from the strange character in c4
 if (in1 != c5)
   cout << "in1==c5 (in1=" << in1 << "  c5="<< c5 << " (c4="<< c4 << ")\n";   // in1=20001     c5=33     c4=! (strange character)
 else
  cout << "in1==c5  (c5="<< c5 << ")\n";
}
```

Example of character use

```
char c ='f';
n ='\n';
char l = 'a';
((l >= 'a')&&(l <= 'z'))

l += 3;
l--;
l-='a'-'A';
```

// l = 'a' (ASCII 97)

//test: is l a lowercase letter?

// l = 'a'+3 = 97+3 = 100 = 'd'

// l = 100 – 1 = 99 = 'c'

// l = 99 – (97 – 65) = 99 – (32) = 67 = 'C'

0	NUL	1	^A	2	^B	3	^C	4	^D	5	^E	6	^F	7	^G
8	^H	9	^I	10	^J	11	^K	12	^L	13	^M	14	^N	15	^O
16	^P	17	^Q	18	^R	19	^S	20	^T	21	^U	22	^V	23	^W
24	^X	25	^Y	26	^Z	27	^[	28	^\	29	^]	30	^^	31	^-
32	SP	33	!	34	"	35	#	36	$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	|	125	}	126	~	127	DEL

Note

- It is possible to apply the arithmetic operators to the objects of type char

Example of use of char:

{ ../L3-4_EXAMPLE_TYPES/char.cc}

The *sizeof* operator

It returns the integer that represents the dimension in **bytes**

- The sizeof operator can be used as prefix for:
  - A variable, e.g., sizeof x
  - A constant, e.g., sizeof 'a'
  - The name of a type, e.g., sizeof double
- The sizeof operator can be used with and without parenthesis, with variables
  - sizeof(x) // Ok
  - sizeof(double) // Ok
  - sizeof(int) // Ok
  - ~~sizeof int~~ // Not Ok
- It is applicable to expressions of any types (not only to the fundamental types)

Example of sizeof applied to the types:

{ ../L3-4_EXAMPLE_TYPES/sizeof.cc}

Example of sizeof applied to the expressions:

{ ../L3-4_EXAMPLE_TYPES/sizeof2.cc}

Mixed operations and type conversion

- Often different types of operands are used in the same expression, or a value of a different type is assigned to a variable
- In any mixed operation it is always needed a conversion of type that can be implicit or explicit

- E.g.,
  - **int** price = 275000;
  - **double** weight = 0.3;
  - **int** cost = price * weight

Implicit type conversions

- Implicit type conversions are done by the compiler
- The most common implicit conversions occur:
  - in the evaluation of numeric expressions, operands are converted to the type of that of larger size
  - in the assignment, an expression is always converted to the type of the variable
- E.g.,
  - float x = 3; //equivalent to: x = 3.0 (i.e., not 3)
  - int y = 2*3.6; //equivalent to: y = 7 (i.e., not 7.2)

Example of implicit conversions:

{ ../L3-4_EXAMPLE_TYPES/conversion_err.cc}

{ ../L3-4_EXAMPLE_TYPES/conversion_corr.cc}

Explicit type conversions

- The programmer can explicit request a type conversion: from a type to another one (named casting)
- Two notations exist
  - Prefix
    - E.g., int x = (int) 3.14;
  - Functional
    - E.g., double f = double(3)

Conversions of numeric types

- Promotion: conversion from one type to a similar one
  - short ⇒ int ⇒ long ⇒ long long,
  - float ⇒ double ⇒ long double
  - It guarantees to maintain the same value
- Conversion between compatible types
  - Conversion from real to integer: the value is truncated
    - int x = (int) 4.7 ⇒ x==4,
    - int x = (int) -4.7 ⇒ x==-4
  - Conversion from integer to real: the value can lose precision
    - float y = float(2147483600); // 2^{31}-48
    - ⇒ y == 2147483600.0 // 2^{31}

Example of conversions between numeric types:

{ ../L3-4_EXAMPLE_TYPES/conversion_mixed.cc}

Example of conversions between real types:

{ ../L3-4_EXAMPLE_TYPES/conversion_real.cc}

For additional details:

http://www.cplusplus.com/doc/tutorial/typecasting